线上赛任务二报告

1. 技术任务要求
2. 运行环境
   2.1 服务端
   2.2 客户端
3. 流程分析
   3.1 功能简介
   3.2 通讯的建立
   3.3 信息的传输
   3.4 客户端监控
   3.5 命令的传达
   3.6 程序的终止
4. 原理与代码分析
   4.1 信号与槽
      4.1.1 Qt中的信号与槽
      4.1.2 本任务中实现的信号与槽
   4.2 通讯协议的制定
      4.2.1 数据头
      4.2.2 编码格式
      4.2.3 结构体打包与解包
         4.2.3.1 结构体定义
         4.2.3.2 python中的struct.pack()和struct.unpack()
         4.2.3.3 本任务中实现的结构体打包与解包
   4.3 图像获取与展示
      4.3.1 验证身份
      4.3.2 传输图像
      4.3.3 展示图像
   4.4 多线程处理
      4.4.1 Thread类

哈工大威海创新平台 - 718-王硕(22969079)

4.4.2 run()的重写

4.4.3 线程的创立与运行

4.5 Qt调用python3

4.5.1 多线程

4.5.2 WinExec()函数

4.5.3 WinExec()与system()的选择

4.5.4 本任务中实现的WinExec()调用

5. 成果展示

1. 技术任务要求

- 上位机开发工具指定为Visual Studio或者Qt；
- 在Windows上创建客户端，ROS上创建服务器端；
- 利用上位机开发工具建立人机交互界面，最终实现Windows端“哈喽,轻舟机器人!”“Hello, World!”“1024 1024 1024”内容与ROS端互传。

2. 运行环境

2.1 服务端

- 使用**Ubuntu20.04**作为系统环境
- 使用**Python3.7**作为软件开发环境
- 程序运行在**ROS** server节点环境中

2.2 客户端

- 使用**Win10**作为系统环境
- 使用**Qt Creator 5.14.2** 作为软件的集成开发环境
- 兼有**Python3.7**编写的python文件作为外部调用

3. 流程分析

3.1 功能简介

以客户端为例。服务端为ROS小车，无需可视化页面，所有功能均与客户端对应



①：程序图标和名称

②：客户输入服务器IP与端口以实现TCP/IP通讯

③：连接按钮，在已连接状态时不可触发

④：断开连接按钮，在未连接状态时不可触发

⑤：监控按钮，在点击后首先会向客户端发送视频传输请求，客户端默认同意，随后程序会调用预先写好的camera.py或camera.exe(经pyinstaller打包)，触发服务端的子线程进行视频传输以更好监控小车动向，同时多线程下不会干扰客户端与服务端的消息交互

⑥：监视窗，使用opencv-python库实现，会以30帧的帧率传输分辨率为480*640的视频，按esc键即可退出，在保持连接时可以点击监控按钮⑤再次打开

⑦：信息输入框，目前的设计为简单的学舌，发送的信息经服务端判定为message后会被以原文发回，也可设计为对特定讯息作出不同反应

⑧：发送消息按钮，点击后信息输入框清空，为消息加入message对应数据头后发送给服务端

⑨：消息记录窗，记录连接操作和发送接收消息的时间

⑩：清屏按钮，清空消息记录窗

⑪：连接标志，此标签会显示是否已连接到服务器

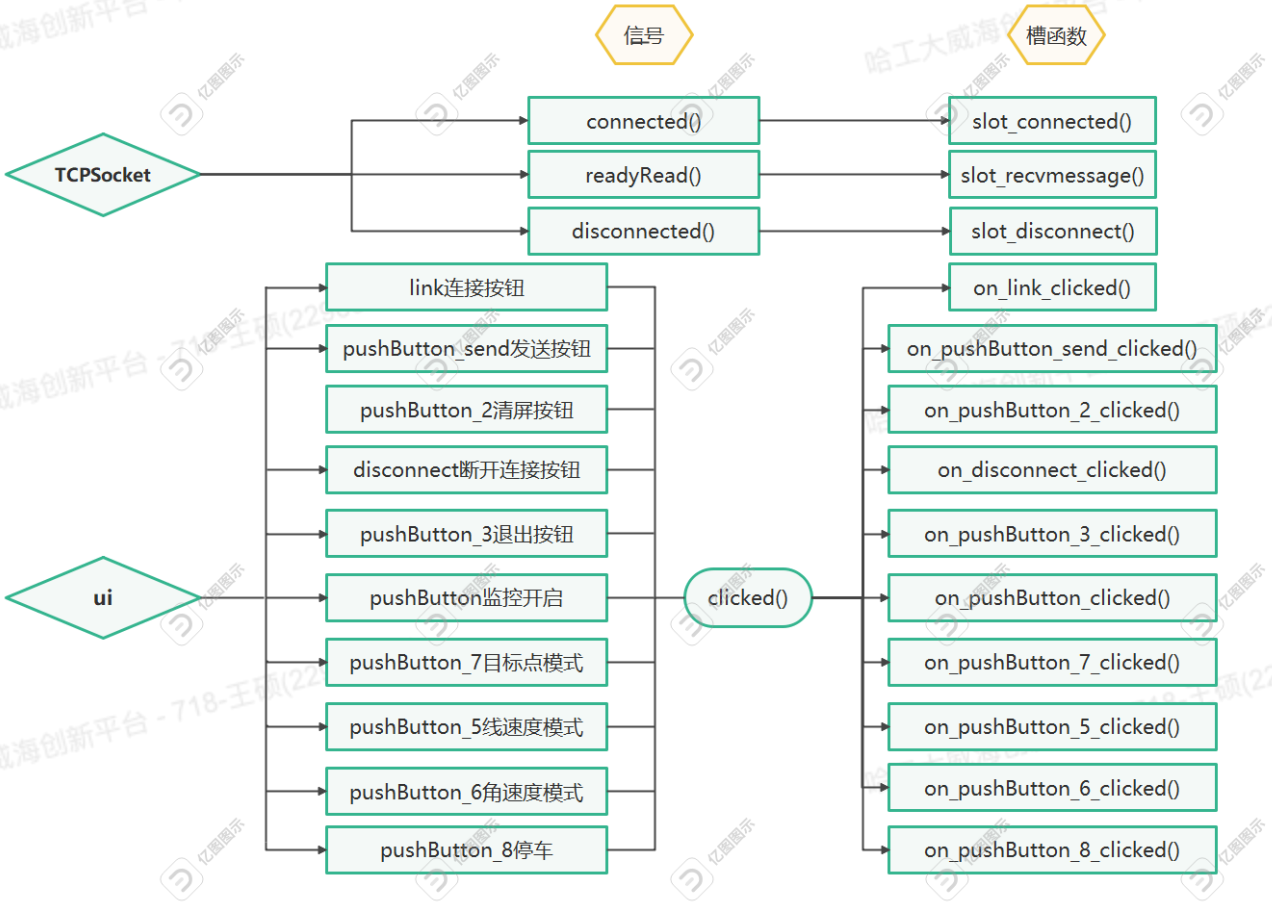
⑫：命令输入栏，有三种命令模式，分别是目标点模式：设定目标点由小车自主导航前往；线速度模式：设定前轮转向角与车轴心线速度；角速度模式：设定阿克曼运动下旋转的半径与角速度；目前服务器端只是接收信息并解包输出在终端，后续线下赛会配合小车上的单片机对命令做出执行

⑬：命令发送按钮，三种命令方式对应三个按钮，同时也对应三个不同的数据头，此外还可以发送刹车命令

⑭：结束程序按钮，在断开与服务端连接时点击可以关闭客户端，在保持与服务端连接时点击会给服务端发送终止命令，服务端与客户端均关闭

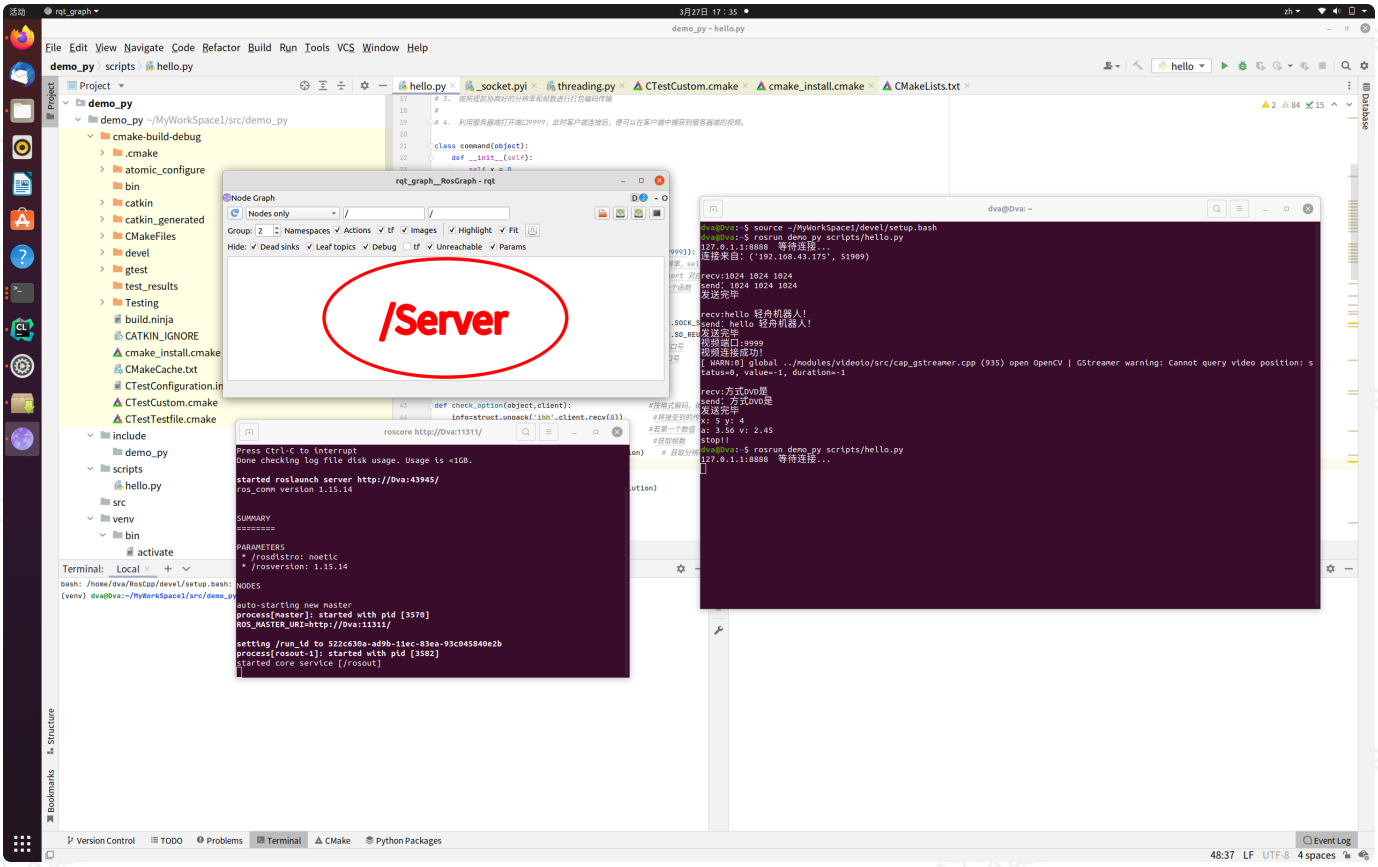
3.2 通讯的建立

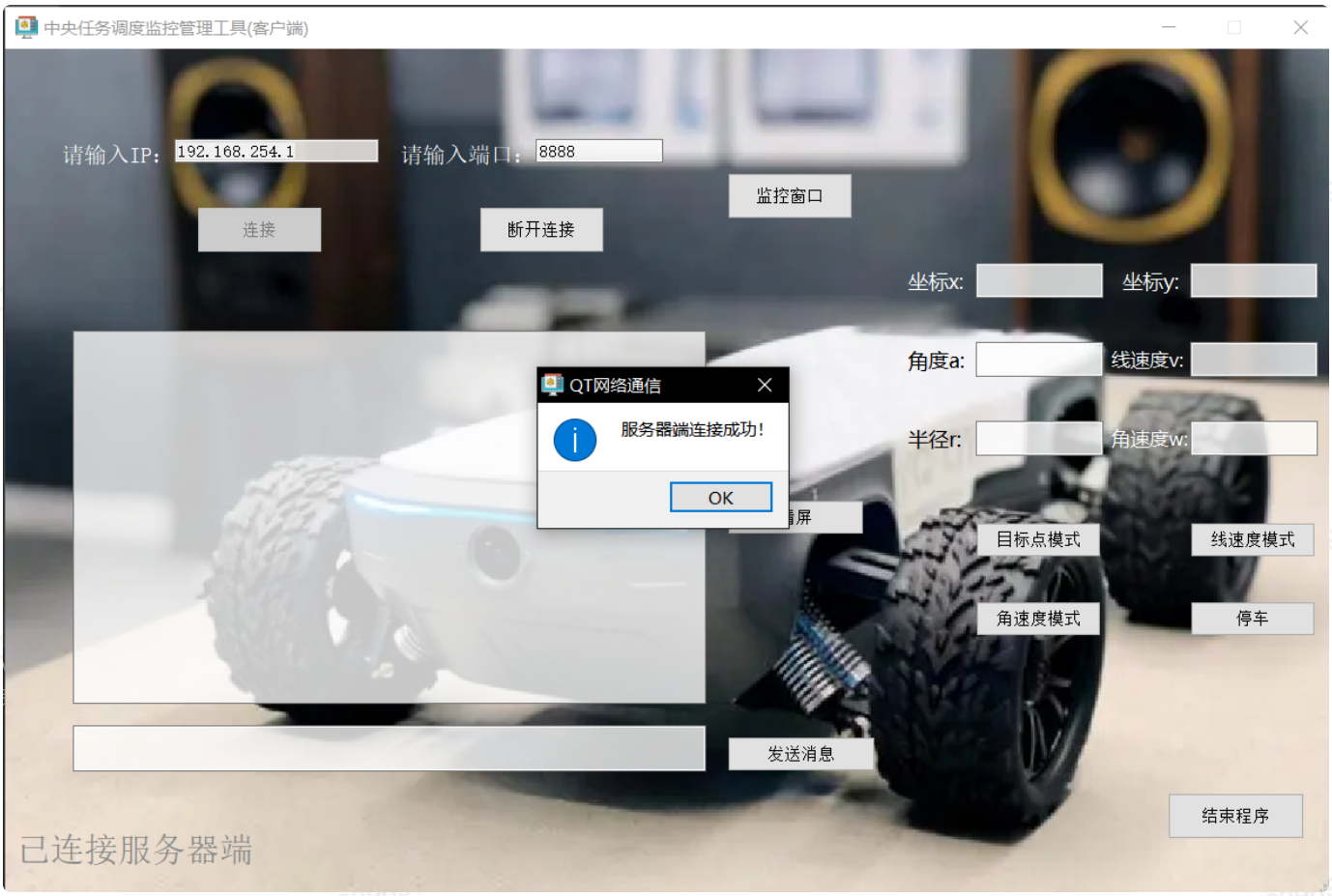
得益于Qt优秀的**槽与信号**机制，我们在客户端建立起信号与槽函数的连接，槽，信号与按键槽函数的链接关系如下图(作图软件有水印，不影响查看)：



经由**QTCPSocket**这个Qt5预定义的通讯类，我们可以轻松的与其他支持TCP协议的机器进行连接，且QTcpSocket中的信号会触发**与之链接的槽函数**，进而在连接完成后做好发送、接收与断连、退出等工作的准备。

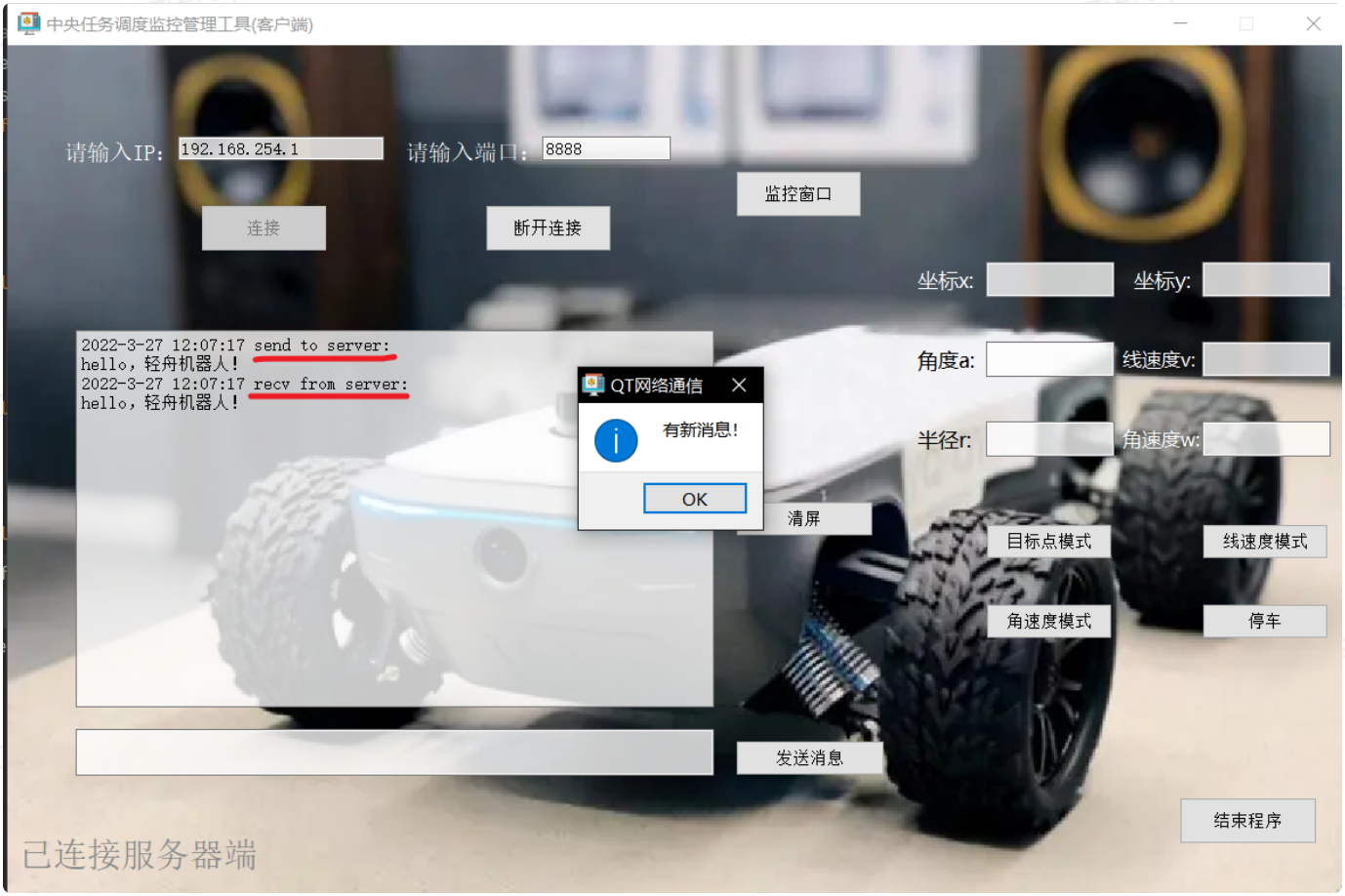
哈工大威海创新平台 - 718-王硕(22969079)


/Server


中央任务调度监控管理工具(客户端)
请输入IP：
192.168.254.1
请输入端口：
8888
连接
断开连接
监控窗口
坐标x:
坐标y:
角度a:
线速度v:
半径r:
角速度w:
QT网络通信
服务器端连接成功!
OK
目标点模式
线速度模式
角速度模式
停车
发送消息
结束程序
已连接服务器端

3.3 信息的传输

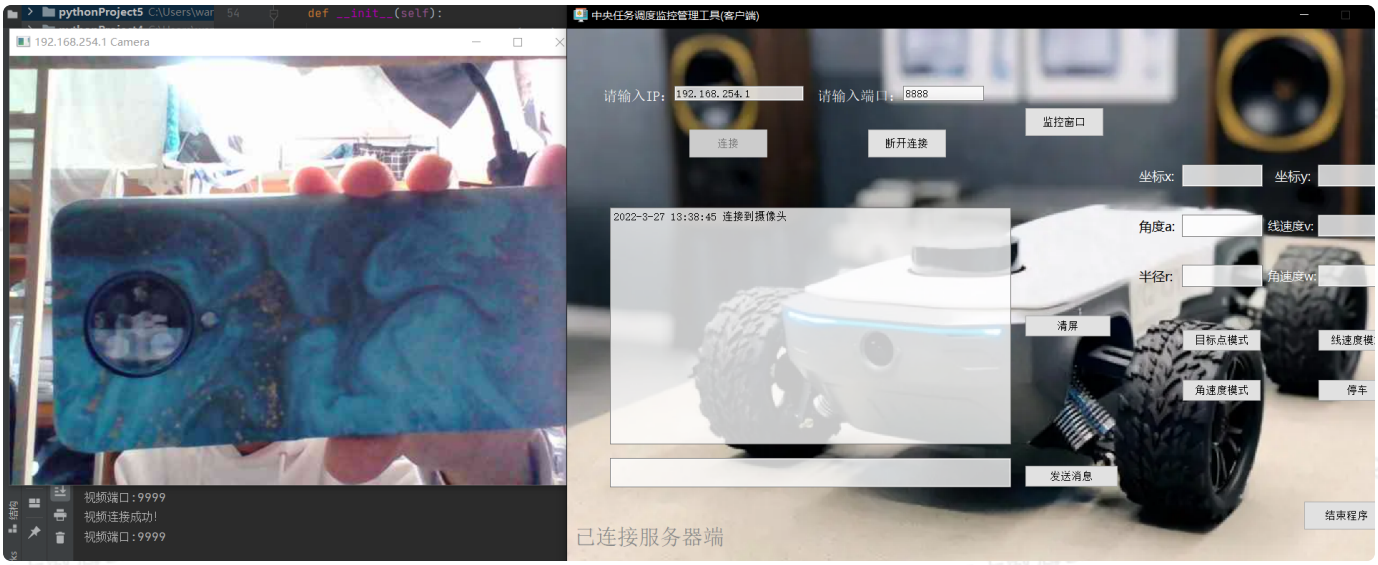
当用户在**信息输入框**输入普通信息并点击**发送消息按钮**后，发出的信号会触发on_pushButton_send_clicked()函数，此函数首先将**信息输入框**的信息加上**数据头**，随后将数据包以**UTF-8编码**后写入TCPSocket的发送缓存区，等待自动发送，随后删除**信息输入框**的内容，服务器端在收到消息并**解码**后会检查**数据头**，验证为普通消息后，会将原文再次**编码**发出，客户端收到消息后解码并显示在消息记录窗口，同时弹窗提醒。



3.4 客户端监控

作为附加功能，为了加快工期，我们选择了**Qt调用python程序**的模式完成监控功能(此程序也经**pyinstaller**打包为了**exe文件**，也可直接调用exe)，同时**服务端**写成**子线程**，保证了**服务端**仍是**一个程序**来**完成所有功能**。点击**监控按钮**后，客户端会给现在连接到的服务器端口发送**视频连接请求**，随后使用**WinExec命令**"python camera.py "+(IP).toStdString()调用外部程序，其中(IP).toStdString()意为将服务器IP作为字符加入到命令中作为程序调用的**传入值**，同时服务器开启**子线程**，**子线程**中会新建一个socket对象，绑定9999端口来进行视频传输，双方确定好**分辨率**、**帧率**等视频传输规范后即由服务器调用摄像头并使用opencv-python模块对图像进行读取并使用numpy模块进行编码后发送，客户端接收后

解码并展示，客户端退出后，服务端也会终止线程，等待下次连接



3.5 命令的传达

命令分为**坐标点模式**、**线速度模式**和**角速度模式**，坐标模式发送两个**int**类型数代表**坐标x**和**坐标y**；线速度模式发送两个**float**类型数，**角度a**和**线速度v**，分别代表小车转弯角度和轴线速度；角速度模式同样发送两个**float**类型数，**半径r**和**角速度w**，分别代表小车转弯半径和角速度，此外，停车命令会发送**stop!!**标志提醒小车停车。目前服务端仅将不同命令下的不同数据类型解码并输出在终端，后期会与任务一的工作融合，将命令信息下放到单片机进行执行

哈工大威海创新平台 - 718-王硕(22969079)




```
连接来自：('192.168.254.1', 62998)
x: 22 y: 77
a: 6.9 v: 5.9
r: 8.09 w: 4.98
stop!!
```

3.6 程序的终止

一般情况下，服务器端父线程处于while(1)循环中，使用listen**阻塞**进程**等待客户端连接**，连接后同样有一个while(1)循环，**等待客户端消**息，若为**视频连接请求**则开启子线程，同时继续while循环，若收到**断开连接请求**则退出此循环，关闭socket并回到上层循环。回到上层循环后会再次初始化socket关键字，继续listen客户端连接；而若在**连接有客户端**的条件下收到**结束程序请求**，服务端会在关闭socket套接字后结束程序并退出。

4. 原理与代码分析

4.1 信号与槽

4.1.1 Qt中的信号与槽

信号槽是 Qt 框架引以为豪的机制之一。所谓信号槽，实际就是观察者模式。**当某个事件发生之后**，比如，按钮检测到自己被点击了一下，**它就会发出一个信号（signal）**。这种发出是没有目的的，类似广播。**如果我们希望某个对象对这个信号感兴趣，就可以使用连接（connect）函数**将**想要处理的信号和自己的一个函数（称为槽（slot））绑定**来处理这个信号。也就是说，**当信号发出时，被连接的槽函数会自动被回调**。这就类似于：当发生了感兴趣的事件，某一个操作就会被自动触发。

信号和槽是Qt特有的信息传输机制，是Qt设计程序的重要基础，它可以让互不干扰的对象建立一种联系。

槽的本质是类的成员函数，其参数可以是任意类型的。和普通C++成员函数几乎没有区别，它可以是虚函数；也可以被重载；可以是公有的、保护的、私有的、也可以被其他C++成员函数调用。唯一区别的是：**槽可以与信号连接在一起，每当和槽连接的信号被发射的时候，就会调用这个槽。**

4.1.2 本任务中实现的信号与槽

信号与槽的对应关系在3.2处已有提及，按键点击信号在编写槽函数时会**默认绑定**，此处仅展示手动connect的信号与槽

C++ 复制代码

```
//定义槽信号
//成功连接服务器的connected()信号连接到slot_connected()
connect(TCPSocket,SIGNAL(connected()),this,SLOT(slot_connected()));
//有新数据到达时的readyread()信号连接到slot_recvmessage()
connect(TCPSocket,SIGNAL(readyRead()),this,SLOT(slot_recvmessage()));
//与服务器断开连接的disconnected()信号连接到slot_disconnect()
connect(TCPSocket,SIGNAL(disconnected()),this,SLOT(slot_disconnect()));
```

4.2 通讯协议的制定

4.2.1 数据头

由于客户端发送数据频次不高，故服务器一次从socket缓存区读取1024位数据并解码，各种不同的信息都有**6位**字符作为数据头，单独的请求也均由**6位**字符组成，以方便服务器**识别**并**作出反应**，数据头对应如下表

哈工大威海创新平台 - 718-王硕(22969079)

end!!!	结束程序请求
___end	断开连接请求
camera	连接视频请求
__cmd1	位置命令数据头
__cmd2	线速度命令数据头
__cmd3	角速度命令数据头
stop!!	刹车命令
mesage	普通消息数据头

4.2.2 编码格式

python中socket通信encode默认使用UTF-8编码，故**服务端**和**客户端**统一使用**UTF-8编码**。

关于UTF-8编码

广为流传的美国标准信息交换代码（American Standard Code for Information Interchange，**ASCII**）仅为美国英语通信所设计，它由**128个**字符组成，包括大小写字母、数字0-9、标点符号、非打印字符（换行符、制表符等4个）以及控制字符（退格、响铃等）组成，当处理带有音调标号（形如汉语的拼音）的亚洲文字时就会出现问题。

但是如果把汉语、日语和越南语的一些相似的字符结合起来，在不同的语言里，使不同的字符代表不同的字，这样只用**2个字节**就可以编码地球上几乎所有地区的文字。因此，出现了**UNICODE编码**。

UTF-8表示**最小单位1字节**的UNICODE。UTF-8与字节序无关（无需BOM），同时**兼容**ASCII编码，使得UTF-8编码成为现今互联网信息编码标准而被**广泛使用**。

以普通消息的发送与接收为例：

客户端发送：

```cpp
if(!sendMessage.isEmpty())
{
    //发送消息到服务器
    this->TCPSocket->write((QString("mesage")+sendMessage).toUtf8());
//  this->TCPSocket->waitForBytesWritten();
    //本地显示发送的消息
    QString localDispalyMessage = QDateTime::currentDateTime().toString("yyyy-M-dd hh:mm:ss ") +tr("send to server: \n") + sendMessage;
    ui->textBrowser->append(localDispalyMessage);
}
```

服务端接收并回复：

```python
data = skt.recv(1024)
...
    print("\nrecv:" + data[6:].decode() + "\nsend： " + data[6:].decode())
    skt.send(data[6:])
    print("发送完毕")
```

客户端接收回复：

```cpp
//接收来自服务器的消息
QByteArray byteArray = this->TCPSocket->readAll();
QString recvMessage = QDateTime::currentDateTime().toString("yyyy-M-dd hh:mm:ss ") + tr("recv from server: \n") + byteArray ;
ui->textBrowser->append(recvMessage);
QMessageBox::information(this, "QT网络通信", "有新消息！");
```

4.2.3 结构体打包与解包

对于python中有struct模块，可以将C语言中传送的struct进行解包，同时也可以将多个数据打包为struct后发送，直接由C语言中的结构题变量承接，而无需逐个数据接收

4.2.3.1 结构体定义

客户端：

struct command C++ 复制代码

```C++
struct command
{
    int x,y;
    float a,v,r,w;
    command()
    {
        this->x = 0;
        this->y = 0;
        this->a = 0.0;
        this->v = 0.0;
        this->r =0.0;
        this->w = 0.0;
    }
};
```

服务端：

class command(object) Python 复制代码

```Python
class command(object):
    def __init__(self):
        self.x = 0
        self.y = 0
        self.a = 0.0
        self.w = 0.0
        self.r = 0.0
        self.v = 0.0
```

服务端以**类**实现**结构体**功能，两端在定义结构体时都会调用构造函数进行初始化

4.2.3.2 python中的struct.pack()和struct.unpack()

严格来说，python中没有结构体这种变量类型，其功能几乎均被**class**所取代，python 中的**struct模块**主要是用来处理与C交互时出现的**结构化数据**的，当然python间通讯也可以使用这样的打包与解包来传输结构化数据。

读入时先转换为Python的**字符串**类型，然后再转换为Python的结构化类型，比如**元组(tuple)**。一般输入的渠道来源于文件或者网络的二进制流。

在**打包**过程中，主要用到了一个**格式化字符串(format strings)**，用来规定转化的方法和格式。python依据格式字符串的要求将多个参数的值按顺序进行一层包装，包装的方法由**fmt**指定，被包装的参数必须**严格符合fmt**。最后返回一个包装后的字符串。

解包返回一个由**解包数据(string)**得到的一个**元组(tuple)**, 即使仅有一个数据也会被解包成元组。其中**len(string)** 必须等于 **calcsize(fmt)**，这里面涉及到了一个calcsize函数。struct.calcsize(fmt)：这个就是用来计算fmt格式所描述的结构的大小。

格式字符串(format string)由一个或多个**格式字符(format characters)**组成，对于这些**格式字符**的描述如下:

（实测在ROS节点中使用的数据类型与**Python** Type不同，需符合C Type）

哈工大威海创新平台 - 718-王硕(22969079)

Format	C Type	Python type	Standard size
x	pad byte	no value	–
c	char	string of length 1	1
b	signed char	integer	1
B	unsigned char	integer	1
?	_Bool	bool	1
h	short	integer	2
H	unsigned short	integer	2
i	int	integer	4
I(大写i)	unsigned int	integer	4
l	long	integer	4
L	unsigned long	integer	4
q	long long	integer	8
Q	unsigned long long	integer	8
f	float	float	4
d	double	float	8
s	char[]	string	1
p	char[]	string	–
P	void *	integer	–

4.2.3.3 本任务中实现的结构体打包与解包

以线速度命令的发送为例

客户端：

```cpp
QString s1 = ui->lineEdit_a->text();//读取输入值
QString s2 = ui->lineEdit_v->text();
command t;
t.a = s1.toFloat();//将输入值写入临时结构体
t.v = s2.toFloat();
QByteArray dataArray = QByteArray::fromRawData((char*)&t, sizeof(t));//将结构体转为字节流格式
this->TCPSocket->write(QString("__cmd2").toUtf8());//发送数据头
this->TCPSocket->write(dataArray);//发送字节流
```

服务端：

```cpp
elif message == "__cmd2":
    com = command()
    _,_,com.a,com.v,_,_ = struct.unpack("2i4f", data[6:])# 从缓冲区第6位开始解包
    print("a:",round(com.a,2),"v:",round(com.v,2))# 保留2位小数
```

从代码可以看出，服务端的**格式化字符串**为**"2i4f"**，由**4.2.3.2中表**可知此结构体由**2个int类型**和**4个float**类型数据组成，与我们的定义相符

4.3 图像获取与展示

4.3.1 验证身份

在客户端发送视频请求后，会与服务端的9999建立一个新通信，通信建立后客户端发送**分辨率**和**帧率**，服务端**验证**后便开始获取并传输图像。

哈工大威海创新平台 - 718-王硕(22969079)

check_option(object,client) Python 复制代码

```python
def check_option(object,client):                    #按格式解码，确定帧数和分辨率，定义一个函数
    info=struct.unpack('ihh',client.recv(8))        #将接受到的传输帧数，分辨率解码
    if info[0]>888:                                 #若第一个数值>888
        object.img_fps=int(info[0])-888             #获取帧数
        object.resolution=list(object.resolution)    # 获取分辨率
        object.resolution[0]=info[1]
        object.resolution[1]=info[2]
        object.resolution = tuple(object.resolution)
        return 1
    else:
        print("数据错误！")
            return 0
```

4.3.2 传输图像

图像的获取是用到了opencv-python模块的函数

哈工大威海创新平台 - 718-王硕(22969079)

run()　　Python　　复制代码

```
 camera = Carame_Accept_Object()  # 执行这个类
        f = True
        while (f):
            f = self.flag
            client, D_addr = camera.server.accept()  # 服务器接受到来自客户端的连接，收到客户端的信息为
            if (check_option(camera, client) == 0):
                return
            video = cv2.VideoCapture(0)  # 从摄像头中获取视频
            img_param = [int(cv2.IMWRITE_JPEG_QUALITY), camera.img_fps]  # 设置传送图像格式、帧数
            while (f):
                f = self.flag
                time.sleep(0.1)  # 推迟线程运行0.1s
                _, camera.img = video.read()  # 读取视频每一帧
                camera.img = cv2.resize(camera.img, camera.resolution)  # 按要求调整图像大小(resolution必须为元组)
                _, img_encode = cv2.imencode('.jpg', camera.img, img_param)  # 按格式生成图片
                img_code = numpy.array(img_encode)  # 转换成数组
                camera.img_data = img_code.tobytes()  # 生成相应的字符串
                try:
                    client.send(struct.pack("ihh", len(camera.img_data), camera.resolution[0],
                                            camera.resolution[1]) + camera.img_data)  # 按照相应的格式进行打包发送图片
                except:
                    video.release()  # 释放资源
                    return
        camera.server.close()
```

line8：**cv2.VideoCapture(0)**在服务器端调用相机，参数为0默认调用电脑内置摄像头

line13：**video.read()** 函数读取视频每一帧，第一个返回值用于标示读取是否成功，第二个返回值为opencv中的Mat类型的图像

line14：**cv2.resize(camera.img, camera.resolution)**将读取到的图像按照客户端发来的分辨率重新组织大小

line15：**cv2.imencode('.jpg', camera.img, img_param)**将传入的图像按**jpg格式编码**

line16：**numpy.array(img_encode)**将编码后的图像转换为numpy中的**数组**类型

line17：**img_code.tobytes()**生成生成相应的字符串准备发送

line19–20：**client.send(struct.pack("3i", len(camera.img_data), camera.resolution[0], camera.resolution[1]) + camera.img_data)**又一次用到pack函数，将要发送数据的长度，两个分辨率

以int，short，short形式发送，随后发送字节串

4.3.3 展示图像

客户端接收到服务端数据后，首先**解包**出该帧图像的**长度**和**分辨率**，随后按**字节数**读取数据

```python
info=struct.unpack("lhh",self.client.recv(8))                    #将接收到的8位字节解包
buf_size=info[0]                                                 #获取读的图片总长度
if buf_size:
    try:
        self.buf=b""                    #代表bytes类型
        temp_buf=self.buf
        while(buf_size):                #读取每一张图片的长度
            temp_buf=self.client.recv(buf_size)
            buf_size-=len(temp_buf)
            self.buf+=temp_buf          #获取图片
            data = numpy.frombuffer(self.buf, dtype='uint8')    #按uint8转换为图像矩阵
            self.image = cv2.imdecode(data, 1)                  #图像解码
            cv2.imshow(self.name, self.image)                   #展示图片
```

line11：**numpy.frombuffer(self.buf, dtype='uint8')**将收到的二进制数据按uint8转换为图像矩阵
line12：**cv2.imdecode(data, 1)**将矩阵以原方式解码为**opencv**中的**Mat**类型的图像
line13：**cv2.imshow(self.name, self.image)**展示图像，窗口名称为：`self.name=self.addr_port[0]+" Camera"`，其中`camera.addr_port[0]=sys.argv[1]`，**sys.argv**为**sys**模块下的函数其为一个**字符串列表**，列表第**0**个元素为**程序名字**，列表第**i**个元素即为调用程序时传入的**第i个参数**，具体调用时传入了什么我们会在**4.5 Qt调用python3**再做展开

4.4 多线程处理

对于服务端，只有一个python程序，我们使用了双线程来处理消息/命令收发和视频传输，其中使用了**threading**模块，由于任务较简单，我们仅使用了**threading**模块中的**Thread**类

4.4.1 Thread类

Thread类内的对象与功能如下：

对象	描述
name	线程名（属性）
ident	线程标识符（属性）
daemon	线程是否是守护线程（属性）
init()	实例化一个线程对象
start()	开启线程
run()	定义线程功能的方法（通常在子类中被应用开发者重写）
Barrier	创建一个障碍，必须达到指定数量线程才开始运行

4.4.2 run()的重写

在服务器端，run()函数如下：

哈工大威海创新平台 - 718-王硕(22969079)

run()

Python | 复制代码

```python
def run(self):
    camera = Carame_Accept_Object()  # 执行这个类
    f = self.flag
    while (f):
        f = self.flag
        client, D_addr = camera.server.accept()  # 服务器接受到来自客户端的连接，收到客户端的信息为
        if (check_option(camera, client) == 0):
            return
        video = cv2.VideoCapture(0, cv2.CAP_DSHOW)  # 从摄像头中获取视频
        img_param = [int(cv2.IMWRITE_JPEG_QUALITY), camera.img_fps]  # 设置传送图像格式、帧数
        while (f):
            f = self.flag
            time.sleep(0.1)  # 推迟线程运行0.1s
            _, camera.img = video.read()  # 读取视频每一帧
            camera.img = cv2.resize(camera.img, camera.resolution)  # 按要求调整图像大小(resolution必须为元组)
            _, img_encode = cv2.imencode('.jpg', camera.img, img_param)  # 按格式生成图片
            img_code = numpy.array(img_encode)  # 转换成数组
            camera.img_data = img_code.tobytes()  # 生成相应的字符串
            try:
                client.send(struct.pack("ihh", len(camera.img_data), camera.resolution[0],
                                        camera.resolution[1]) + camera.img_data)  # 按照相应的格式进行打包发送图片
            except:
                self.flag = False
                video.release()  # 释放资源
                return
    camera.server.close()
```

本部分代码在**4.3.2 传输图像**已进行分析，不再赘述

4.4.3 线程的创立与运行

```python
if message == "camera":
    thread1 = MyThread()
    if not thread1.is_alive():# 防止重复开启线程导致程序崩溃
        thread1.start()
        print("视频连接成功！")
```

在主线程的**while**循环中，若接到视频请求**"camera"**，服务端会创建一个**Thread对象**并开启，此进程便开始运行**run()**函数，客户端关闭视频窗口后，**视频传输的socket连接**断开，**run()**函数触发异常处理，**退出循环**并**关闭套接字**，等待下次线程再次被实例化

4.5 Qt调用python3

4.5.1 多线程

调用同样使用到了多线程来调用外部程序。通过对Qt中的<QThread>头文件继承**QThread**类可以定义自己的线程类：

MyThread

```cpp
class MyThread : public QThread
{
public:
    MyThread();
    void stop();
protected:
    void run();

private:
    volatile bool stopped;
};
```

其中的**stopped**本是线程终止的标志位，但由于线程在调用的程序终止后可以自动结束，故此处没有用上

对于**MyThread线程对象**我们直接定义在头文件的**private**型变量中，当需要连接服务器端摄像头时：

```cpp
if(this->isconnection)
{
        QString conn="camera";
        this->TCPSocket->write(conn.toUtf8());
        thread.start();
        QString camlink = QDateTime::currentDateTime().toString("yyyy-M-dd hh:mm:ss ") + tr("连接到摄像头");
        ui->textBrowser->append(camlink);
}
```

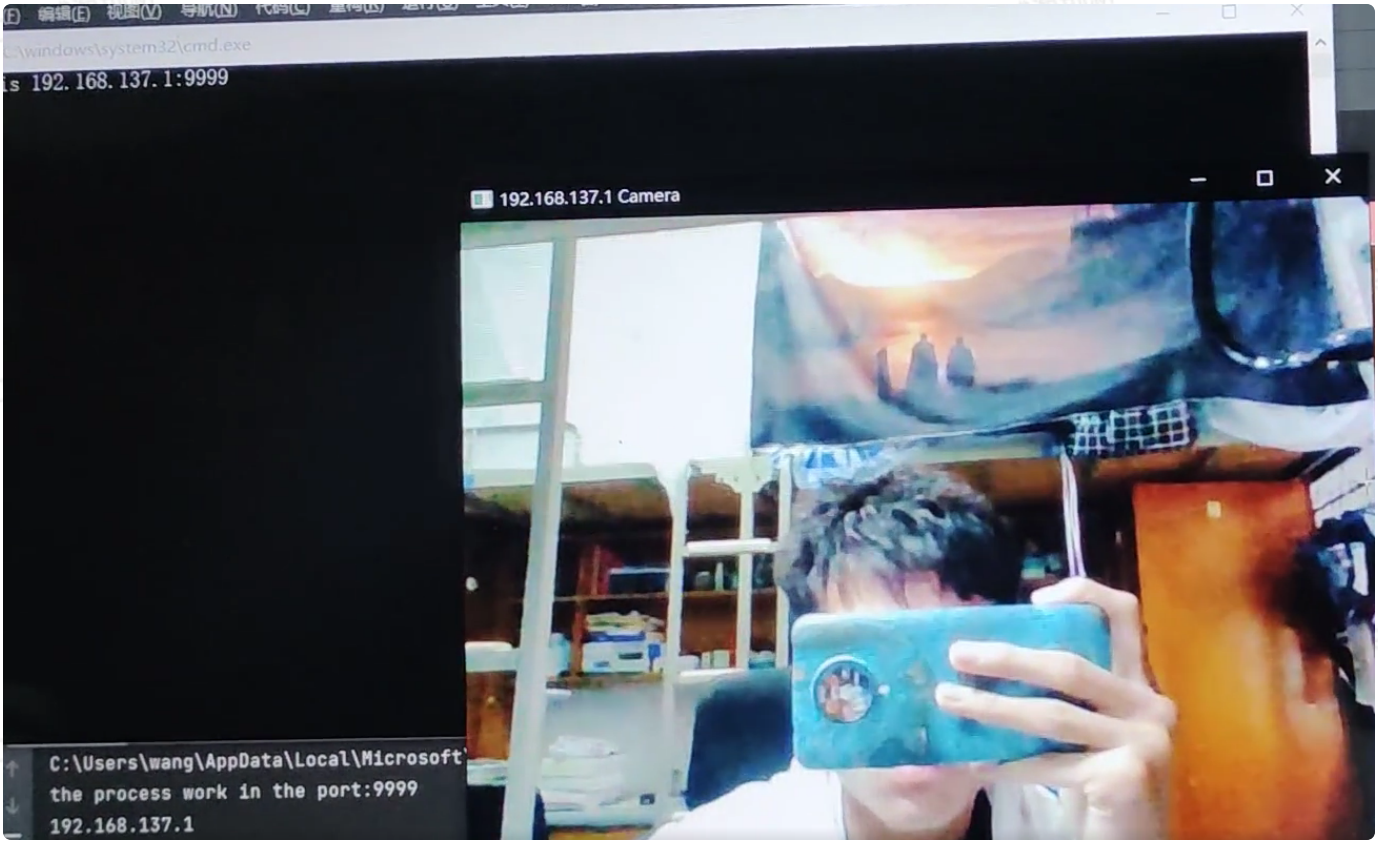
4.5.2 WinExec()函数

WinExec是一个WIN32 API，它的第一个参数必须包含一个**可执行文件名**，在包括了头文件 `#include <windows.h>` 后即可对其传入char *类型的**可执行文件名**和**需要传入的参数**，此函数即可执行此文件并传参。其第二个参数为打开模式，各参数意义如下表

哈工大威海创新平台 - 718-王硕(22969079)

参数	意义
SW_HIDE、0	隐藏窗口并激活另一个窗口。
SW_SHOWNORMAL SW_NORMAL、1	激活并显示一个窗口。如果窗口被最小化或最大化，系统会将其恢复到原来的大小和位置。应用程序应在第一次显示窗口时指定此标志。
SW_SHOWMINIMIZED、2	激活窗口并将其显示为最小化窗口。
SW_SHOWMAXIMIZED SW_MAXIMIZE、3	激活窗口并将其显示为最大化窗口。
SW_SHOWNOACTIVATE、4	以最近的大小和位置显示窗口。这个值类似于SW_SHOWNORMAL，但窗口没有被激活。
SW_SHOW、5	激活窗口并以其当前大小和位置显示它。
SW_MINIMIZE、6	最小化指定窗口并激活顺序中的下一个顶级窗口。
SW_SHOWMINNOACTIVE、7	将窗口显示为最小化窗口。这个值类似于SW_SHOWMINIMIZED，但窗口没有被激活。
SW_SHOWNA、8	以当前大小和位置显示窗口。这个值类似于SW_SHOW，但窗口没有被激活。
SW_RESTORE、9	激活并显示窗口。如果窗口被最小化或最大化，系统会将其恢复到原来的大小和位置。应用程序在恢复最小化窗口时应指定此标志。
SW_SHOWDEFAULT、10	根据启动应用程序的程序传递给CreateProcess函数的 STARTUPINFO结构中指定的SW_值设置显示状态。
SW_FORCEMINIMIZE、11	最小化一个窗口。只有在最小化来自不同线程的窗口时才应使用此标志。

4.5.3 WinExec()与system()的选择



system()是C库函数，它接受一个**DOS命令**。在一开始，我们使用的**system()**函数，但正如刚刚所说，**system()**函数需要接收一个**DOS命令**，这也就意味着我们必须打开一个**cmd窗口(如上图)**，即便我们调用的程序**没有输出**或者我们将输出**另存到log文件**，在运行程序的时候都有个大黑框，一旦我们关闭这个黑框，监控窗口也会关闭，这样的UI是**不够友好**的，于是我们查询资料后选择了**WinExec()**，其中的SW_HIDE可以**隐藏cmd窗口**并**激活监控窗口**，与我们的需求完美符合

4.5.4 本任务中实现的WinExec()调用

哈工大威海创新平台 - 718-王硕(22969079)

run()

C++ 复制代码

```cpp
void MyThread::run()
{
    std::string s = "camera.exe "+(IP).toStdString();      //可以调用python程序，也可以调用打包好的exe程序
    //std::string s = "python camera.py "+(IP).toStdString();
    const char* result = s.c_str();     //将字符串str变量变为char型数组
    int tail = 0;
    while (result[tail] != '\0') {
            tail++;
    }
    char* real_result = (char*)malloc(tail+1);
    for (int i = 0; i <= tail; i++) {
        real_result[i] = result[i];
    }
    WinExec(real_result, 0);
    //system(real_result);
}
```

由于**WinExec()**的第一个参数是**char *类型的可执行文件名和需要传入的参数**，故为了防止字符串尾的终止符'\0'干扰，需将字符串str变量变为char型数组

此处的**(IP).toStdString()**即为前面**4.3.3 展示图像**提到的调用函数时传入的参数，IP为客户端上用户输入的服务器IP，传给**camera.py**或**camera.exe**

5. 成果展示

链接：https://pan.baidu.com/s/12gNeyr9loh9DAiAzKfjm3w?pwd=1024

提取码：1024

视频中实现了以下内容（红色为拓展内容）：

- 客户端与服务器之间的Socket通讯传递**字符串**
- 客户端与服务器之间的Socket通讯传递**打包的结构体**命令并**解包**
- 客户端调用python程序接收服务器**子线程**传来的**编码视频信息**并**解码展示**
- 客户端与服务端在传递视频的同时在**主线程**使用另一端口正常通讯
- 客户端在**保持连接**的前提下结束程序并使服务端也**正常退出**